CodeOrbit — Web Uygulama Sınıfı Sunumu

> **Real-Time Collaborative Browser IDE**

Slide 1: Proje Tanıtımı

CodeOrbit Nedir?

CodeOrbit, tarayıcıda çalışan, çok kullanıcılı, gerçek zamanlı bir işbirlikçi IDE'dir.

Temel Özellikler:

- **Oda (Room) bazlı çalışma alanları** — Her oda bağımsız bir proje
- **Gerçek zamanlı kod editörü** — Monaco Editor + Yjs CRDT (çakışma yok, aynı anda yazma)
- **VS Code tarzı dosya gezgini** — Hiyerarşik klasör yapısı, sürükle-bırak, içeride yeniden adlandırma
- **Çok dilli kod çalıştırma** — 10 programlama dili desteği
- **JWT + Google OAuth2** kimlik doğrulama
- **Profil yönetimi** — BLOB tabanlı profil fotoğrafı, istatistikler
- **3D açılış sayfası** — Spline + GSAP animasyonları

Problem Çözdüğü: Geliştirici ekiplerin aynı projeyi fiziksel ortamda veya VS Code Live Share gibi eklentiler olmadan, sadece bir tarayıcı üzerinden ortak çalışabilmesi.

Slide 2: Sistem Mimarisi

Genel Yapı

```
┌──────────────────────────────────────────────────────────────┐
│                          BROWSER                             │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐ │
│  │  Thymeleaf   │  │   React 18   │  │  Monaco + xterm.js   │ │
│  │   Shell      │  │   (Vite)     │  │   + WebContainer     │ │
│  └──────────────┘  └──────────────┘  └──────────────────────┘ │
│         │                │                      │            │
│         └────────────────┴──────────────────────┘            │
│                          │                                   │
│                          │                                   │
│              HTTP (JWT)  │  WebSocket (STOMP)                │
└──────────────────────────┼───────────────────────────────────┘
                           │
                           ▼
                ┌──────────────────────┐
                │    Spring Boot 4.0.6 │
                │    (Java 21, Port 8080) │
                │  ┌──────────────────┐│
                │  │ REST Controllers │
                │  │ WebSocket Broker │
                │  │ OAuth2 + JWT     │
                │  └──────────────────┘│
                └──────────────────────┘
                           │
                           ▼
                ┌──────────────────────┐
                │    PostgreSQL        │
                │    (Kalıcı Depolama) │
                └──────────────────────┘
```

Neden Hybrid Rendering?

Sayfa	Teknoloji	Neden?
`/` (Landing)	React	3D Spline, GSAP animasyonları
`/login`, `/register`	Thymeleaf + Alpine.js	SEO, hızlı yükleme, güvenlik
`/dashboard`	React	Dinamik oda listesi, etkileşim
`/room/{id}`	React	Monaco Editor, terminal, WebSocket
`/profile`	React	Profil formu, resim upload

Slide 3: Veritabanı Tasarımı (DB Schema)

PostgreSQL Tabloları

```sql
-- 1. USERS (Kullanıcılar)
CREATE TABLE users (
  id                  BIGSERIAL PRIMARY KEY,
  username            VARCHAR NOT NULL UNIQUE,
  email               VARCHAR NOT NULL UNIQUE,
  password            VARCHAR,              -- LOCAL kullanıcılar için
  provider            VARCHAR DEFAULT 'LOCAL', -- LOCAL | GOOGLE | GITHUB
  profile_image       BYTEA,                -- Binary profil fotoğrafı
  profile_image_content_type VARCHAR,
  profile_image_file_name    VARCHAR,
  profile_image_updated_at   TIMESTAMP
);

-- 2. ROOMS (Çalışma Odaları)
CREATE TABLE rooms (
  id         VARCHAR PRIMARY KEY,        -- UUID (örn: a1b2-c3d4-...)
  name       VARCHAR NOT NULL,
  owner_id   BIGINT  NOT NULL,
  created_at TIMESTAMP
);

-- 3. ROOM_MEMBERS (Oda Üyeliği — Çoktan-Çoğa)
CREATE TABLE room_members (
  id         BIGSERIAL PRIMARY KEY,
  room_id    VARCHAR NOT NULL REFERENCES rooms(id),
  user_id    BIGINT  NOT NULL REFERENCES users(id),
  joined_at  TIMESTAMP,
  UNIQUE (room_id, user_id)
);
CREATE INDEX idx_rm_room ON room_members (room_id);
CREATE INDEX idx_rm_user ON room_members (user_id);

-- 4. PROJECT_FILES (Dosya Sistemi — Adjacency List)
CREATE TABLE project_files (
  id           BIGSERIAL PRIMARY KEY,
  room_id      VARCHAR   NOT NULL,       -- Hangi odada?
  file_path    VARCHAR   NOT NULL,       -- Tam yol: src/components/App.jsx
  name         VARCHAR   NOT NULL,       -- Dosya adı: App.jsx
  parent_id    BIGINT    REFERENCES project_files(id), -- Üst klasör (NULL = root)
  file_type    VARCHAR   NOT NULL DEFAULT 'FILE', -- FILE | DIRECTORY
  content      TEXT,                     -- Dosya içeriği (klasörlerde NULL)
  yjs_state    BYTEA,                    -- Yjs CRDT encoded state snapshot
  last_updated TIMESTAMP,
  UNIQUE (room_id, file_path)
);
CREATE INDEX idx_pf_room   ON project_files (room_id);
CREATE INDEX idx_pf_parent ON project_files (parent_id);
CREATE INDEX idx_pf_path   ON project_files (file_path);
```

Tasarım Kararları

Karar	Açıklama
Adjacency List	`parent_id` ile hiyerarşik yapı — ağaç derinliği sınırsız
UUID for Rooms	Oda ID'lerini URL'de paylaşmak için güvenli (tahmin edilemez)
BYTEA for Images	Profil fotoğrafları veritabanında, dosya sistemi karmaşası yok
Room Membership	Ayrı tablo çoktan-çokğa ilişki için — `existsByRoomIdAndUserId` ile hızlı kontrol
Unique (roomId, filePath)	Aynı odada aynı yolda iki dosya olamaz

---

Slide 4: Backend Mimarisi

Katmanlı Yapı

```
┌──────────────────────────────────────┐
│  Controller Layer (REST + WS)        │
│  ├── AuthController      (/api/auth)  │
│  ├── RoomController      (/api/rooms) │
│  ├── ProjectFileController (/api/files)│
│  ├── CodeController      (WS /app/code)│
│  └── ViewController      (Thymeleaf)  │
├──────────────────────────────────────┤
│  Service Layer                       │
│  ├── RoomService       (Oda CRUD + üye)│
│  ├── ProjectFileService (FS + WS yayın)│
│  ├── CodeExecutionService (Sandbox)  │
│  └── CustomUserDetailsService        │
├──────────────────────────────────────┤
│  Repository Layer (Spring Data JPA)  │
│  ├── UserRepository                  │
│  ├── RoomRepository                  │
│  ├── RoomMemberRepository            │
│  └── ProjectFileRepository           │
├──────────────────────────────────────┤
│  Entity Layer                        │
│  ├── User, Room, RoomMember, ProjectFile│
└──────────────────────────────────────┘
```

Güvenlik Akışı

```
Kullanıcı
   │
   ├─→ /login (Thymeleaf form)
   │      └─→ POST /api/auth/login
   │            └─→ JWT token → localStorage
   │
   ├─→ /oauth2/authorization/google
   │      └─→ Google OAuth2
   │            └─→ /api/auth/oauth2/success
   │                  └─→ JWT cookie + redirect /dashboard
   │
   └─→ Her istek: Authorization: Bearer <token>
         └─→ JwtFilter → SecurityContext
```

WebSocket (STOMP) Yapılandırması

```
@Override
public void configureMessageBroker(MessageBrokerRegistry config) {
    config.enableSimpleBroker("/topic");      -- Yayın kanalı
    config.setApplicationDestinationPrefixes("/app"); -- Gelen mesajlar
}

@Override
public void registerStompEndpoints(StompEndpointRegistry registry) {
    registry.addEndpoint("/ws")
            .setAllowedOriginPatterns("*")
            .withSockJS();   -- SockJS fallback
}
```

Kanal	Amaç
`/app/code.send`	Editörde yazılan kodu gönder
`/topic/code/{roomId}`	Odadaki herkese kodu yayınla
`/topic/fs/{roomId}`	Dosya sistemi olaylarını yayınla

Slide 5: Frontend Mimarisi

Teknoloji Yığını

Katman	Teknoloji
Build	Vite 5 (çok girişli: main.jsx + alpine.js + index.css)
Framework	React 18 (fonksiyonel komponentler + hooks)
Routing	react-router-dom (BrowserRouter)
Styling	Tailwind CSS + inline styles
State	useState, useRef, useCallback, custom hooks
3D	@splinetool/react-spline
Animasyon	GSAP + ScrollTrigger
Editör	@monaco-editor/react (VS Code editörü)
Terminal	xterm.js + FitAddon + WebContainer API
WebSocket	@stomp/stompjs + sockjs-client
HTTP	Axios (JWT interceptor ile)
Form UI	Alpine.js (login/register Thymeleaf sayfalarında)

React Bileşen Haritası

```
App.jsx
├── LandingPage (/)           → SplineBackground + GSAP sections
│   ├── HeroSection
│   ├── TechStackSection
│   ├── FeaturesSection
│   ├── ArchitectureSection
│   └── CTASection
│
├── LoginRedirect (/login)    → Backend Thymeleaf'e yönlendirir
├── RegisterRedirect (/register)
├── OAuth2Redirect (/oauth2-redirect)
│
├── Dashboard (/dashboard)    → Oda listesi, oluştur, katıl
│
├── Room (/room/:roomId)      → TAM IDE
│   ├── RoomIDE
│   │   ├── FileTree          → Recursive tree, drag-drop, context menu
│   │   ├── TabBar            → Açık dosya sekmeleri
│   │   ├── Editor            → Monaco + WebSocket senkronizasyonu
│   │   └── TerminalPanel     → xterm.js + WebContainer
│   └── useFileSystem hook    → FS state + STOMP subscriber
│
└── Profile (/profile)        → Profil bilgileri, resim upload
```

Editor Senkronizasyonu (Yjs CRDT — Gerçek Zamanlı)

CodeOrbit artık **Yjs CRDT** kullanarak gerçek zamanlı işbirlikçi düzenleme sağlıyor. Bu, iki kullanıcının aynı dosyada aynı anda yazmasına olanak tanır ve tüm değişiklikler otomatik olarak çakışma olmadan birleşir.

```
// Her dosya için bir Y.Doc
const ydoc = new Y.Doc();
const ytext = ydoc.getText('monaco');

// MonacoBinding — Y.Text ↔ Monaco model otomatik senkronizasyonu
const binding = new MonacoBinding(ytext, model, new Set([editor]), null);

// Yerel değişiklik → STOMP üzerinden yayımlanır
ydoc.on('update', (update, origin) => {
    if (origin === provider) return; // kendi uyguladığımız güncellemeyi atla
    stompClient.publish({
        destination: '/app/code.send',
        body: JSON.stringify({
            roomId, filePath, clientId,
            type: 'yjs-update',
            update: toBase64(update) // base64 Uint8Array
        })
    });
});

// Gelen mesaj → Y.applyUpdate ile CRDT birleştirme
provider.onRemoteMessage = (msg) => {
    if (msg.type === 'yjs-update') {
        Y.applyUpdate(ydoc, fromBase64(msg.update), provider);
    }
};
```

Mesaj Türleri:

Tür	Yön	Açıklama
`yjs-update`	Giden / Gelen	Artımlı CRDT güncellemesi (base64 `Uint8Array`)
`yjs-request`	Giden	Yeni istemci tam durum talep eder
`yjs-offer`	Gelen	Aktif peer (>2s) `encodeStateAsUpdate(doc)` ile yanıt verir
`yjs-full`	Giden	5 saniyede bir periyodik tam durum anlık görüntüsü (DB'ye kaydedilir)

Özellikler:

- **Echo önleme**: `clientId` filtresi — kendi mesajlarımızı görmezden gelir
- **STOMP bağlantı yarış durumu**: WebSocket bağlanmadan önce açılan dosyalar için sağlayıcı `onConnect`'te tembel oluşturulur
- **Çift sağlayıcı koruması**: Bir dosyaya tekrar dönüldüğünde mevcut sağlayıcı yeniden kullanılır
- **DB kalıcılığı**: `yjsState` (BYTEA) hem yeni istemcilerin hızlı yakınsaması hem de otomatik kurtarma için depolanır

---

Slide 6: Spring Boot + Thymeleaf Kullanımı

Hybrid Rendering: Thymeleaf + React

CodeOrbit'te **tek sayfa (SPA)** yerine **çok sayfalı hybrid** yaklaşımı kullanıldı. Thymeleaf statik sayfaları yönetir, React ise yüksek etkileşim gerektiren sayfalara mount olur.

Sayfa Dağılımı

Route	Teknoloji	Neden?
`/`	React	3D Spline sahne, GSAP scroll animasyonları
`/login`	**Thymeleaf + Alpine.js**	Form validation, güvenlik, SEO
`/register`	**Thymeleaf + Alpine.js**	Form validation, güvenlik
`/dashboard`	**Thymeleaf shell → React mount**	Dinamik oda listesi
`/room/{id}`	**Thymeleaf shell → React mount**	Monaco, terminal, WebSocket
`/profile`	**Thymeleaf shell → React mount**	Profil formu, resim upload

ViewController — Spring Boot Route Yönlendirme

```java
@Controller
public class ViewController {

    @GetMapping("/")
    public String index(Model model,
                        @AuthenticationPrincipal UserDetails user) {
        if (user != null) model.addAttribute("username", user.getUsername());
        return "index";   -- index.html (React SPA mount point)
    }

    @GetMapping("/login")
    public String login() {
        return "auth/login";   -- Thymeleaf login formu
    }

    @GetMapping("/dashboard")
    public String dashboard(Model model,
                            @AuthenticationPrincipal UserDetails user) {
        if (user == null) return "redirect:/login";
        model.addAttribute("username", user.getUsername());
        return "dashboard";   -- Thymeleaf shell → React mount
    }

    @GetMapping("/room/{id}")
    public String room(@PathVariable String id, Model model,
                       @AuthenticationPrincipal UserDetails user) {
        if (user == null) return "redirect:/login";
        model.addAttribute("room", roomRepository.findById(id).orElseThrow());
        model.addAttribute("username", user.getUsername());
        return "room";   -- Thymeleaf shell → React IDE mount
    }
}
```

Thymeleaf Layout Sistemi

`layout.html` — Global kabuk (navbar + footer + asset'ler):

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      th:fragment="layout(title, content)">
<head>
    <title th:text="${title} + ' — CodeOrbit'">CodeOrbit</title>

    <!-- Vite build output'u -->
    <link rel="stylesheet" th:href="@{/dist/assets/styles.css}">
    <link rel="stylesheet" th:href="@{/dist/assets/main.css}">
    <script type="module" th:src="@{/dist/assets/alpine.js}" defer></script>
    <script type="module" th:src="@{/dist/assets/main.js}" defer></script>

    <script src="https://unpkg.com/htmx.org@1.9.10"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.12.5/gsap.min.js"></script>
</head>
<body class="bg-[#0D0D11] text-gray-100">
    <div th:replace="~{fragments/navbar :: navbar}"></div>
    <main th:replace="${content}"></main>
    <div th:replace="~{fragments/footer :: footer}"></div>
</body>
</html>
```

`dashboard.html` — Layout'i extend eden sayfa:

```
<html th:replace="~{fragments/layout :: layout(title='Dashboard', content=~{::content})}">
<body>
    <div th:fragment="content" class="w-full h-full flex-grow relative">
        <!-- React Dashboard buraya mount olur -->
        <div id="app-root" class="w-full h-full"></div>
    </div>
</body>
</html>
```

React Mount Mekanizması

`main.jsx` her sayfa yüklendiğinde çalışır ve doğru DOM element'ini bulur:

```
// frontend/src/main.jsx
const appRoot = document.getElementById('app-root');
if (appRoot) {
    ReactDOM.createRoot(appRoot).render(<App />);
}

const landingRoot = document.getElementById('landing-react-root');
if (landingRoot) {
    ReactDOM.createRoot(landingRoot).render(
        <BrowserRouter><LandingPage /></BrowserRouter>
    );
}
```

DOM ID	Kullanıldığı Şablon	React Bileşeni
`app-root`	`dashboard.html`, `room.html`, `profile.html`	`<App />` (Router dahil)
`landing-react-root`	`index.html` (standalone)	`<LandingPage />`

Vite Build → Spring Boot Static

```
// frontend/vite.config.js
export default defineConfig({
    build: {
        outDir: '../src/main/resources/static/dist',
        emptyOutDir: true,
        rollupOptions: {
            input: {
                main: './src/main.jsx',
                alpine: './src/alpine-init.js',
                styles: './src/index.css'
            }
        }
    }
})
```

Build çıktısı Spring Boot `static/` dizinine yazılır:

```
src/main/resources/static/dist/
├── assets/
│   ├── main.js          (React bundle)
│   ├── alpine.js        (Alpine.js bundle)
│   ├── styles.css
│   └── main.css
└── ...
```

Neden Hybrid?

SPA (Pure React)	Hybrid (Thymeleaf + React)
Tek entry point, client-side routing	Server-side routing, SSR avantajı
Login formu JSX'te	Login formu Thymeleaf'te — Alpine.js ile validation
SEO zorluğu	Thymeleaf sayfaları tarayıcıya hazır HTML gönderir
Initial bundle büyük	Sadece gerekli sayfa için React yüklenir

---

Slide 7: Kod Çalıştırma Mimarisi

İki Katmanlı Yaklaşım

Katman	Teknoloji	Kullanım
Backend Sandbox	Temp dizin + ProcessBuilder	Kod çalıştırma (Run butonu)
Frontend WebContainer	@webcontainer/api + xterm.js	Etkileşimli terminal (shell)

Backend Çalıştırma Akışı

```
1. Kullanıcı "RUN" butonuna basar
        |
2. useBackendRunner hook dil tespiti yapar (.py → Python)
        |
3. POST /api/execute → {language, code, fileName, stdin, timeout}
        |
4. CodeExecutionService:
   a. /tmp/codeorbit_{uuid} dizini oluşturur
   b. Kodu dosyaya yazar
   c. Uygun derleyici/interpreter çalıştırır:
      python3, node, npx ts-node, gcc, g++,
      rustc, go run, javac→java, ruby, php
   d. stdout/stderr yakalar (30 sn timeout)
        |
5. Sonuç JSON olarak döner
        |
6. Frontend xterm.js'e doğrudan yazar
```

WebContainer (TerminalPanel)

```
// Tarayıcıda gerçek Node.js çalışma zamanı!
const wc = await WebContainer.boot();
await wc.fs.writeFile('/index.js', 'console.log("Hello")');
const process = await wc.spawn('node', ['index.js']);
// process.output.pipeTo(xtermInput)
```

Gerekli Headerlar (vite.config.js):

```
headers: {
    'Cross-Origin-Embedder-Policy': 'require-corp',
    'Cross-Origin-Opener-Policy': 'same-origin',
}
```

Slide 8: Dağıtım (Deployment)

Docker Multi-Stage Build

```
# Stage 1: Build
FROM maven:3.9.6-eclipse-temurin-21 AS build
WORKDIR /app
RUN apt-get install -y nodejs
COPY frontend/ ./frontend/
RUN npm run build --prefix frontend   → static/dist/
COPY src ./src
RUN mvn clean package -DskipTests   → app.jar

# Stage 2: Runtime
FROM eclipse-temurin:21-jre-jammy
RUN apt-get install -y \
    python3 golang-go default-jdk \
    build-essential nodejs
RUN curl --proto '=https' --tlsv1.2 -sSf \
    https://sh.rustup.rs | sh -s -- -y
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
HEALTHCHECK CMD curl -f http://localhost:8080/login || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Render Cloud Deploy

Adım	Açıklama
1	GitHub repo'sunu Render'a bağla
2	Environment: Docker
3	Env vars gir: `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`, `JWT_SECRET`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`
4	Google Cloud Console'da callback URL: `https://<domain>.onrender.com/login/oauth2/code/google`
5	Otomatik build + deploy

Çevre Değişkenleri

```
PORT=8080
SPRING_DATASOURCE_URL=jdbc:postgresql://host:5432/db
SPRING_DATASOURCE_USERNAME=...
SPRING_DATASOURCE_PASSWORD=...
JWT_SECRET=base64-encoded-secret
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
```

Slide 9: Zorluklar ve Çözümler

Zorluk	Çözüm
Concurrent edit	**Yjs CRDT** — aynı anda yazma tam destekli, otomatik çakışma çözümü. Artımlı güncellemeler STOMP üzerinden yayınlanır.
WebContainer headers	Vite dev server COOP/COEP headerları. Spring Boot prod headerları.
Dosya sistemi senkronizasyonu	Her REST mutasyonu → WebSocket yayını → tüm client'lar optimistik güncelleme
Klasör taşıma döngüsü	`guardCyclicMove()` — hedefin atası olup olmadığını kontrol eder
Rename cascade	JPQL `LIKE` prefix query ile tüm alt öğelerin yolunu toplu güncelle
Çoklu dil desteği	ProcessBuilder ile temp dizin sandbox — her dil için ayrı komut
Görsel tutarlılık	WebContainer + PostgreSQL aynı anda güncellenir (FS op → hem DB hem WC)

Slide 10: Özet ve Teknoloji Yığını

CodeOrbit Teknoloji Özeti

Katman	Teknolojiler
Backend	Java 21, Spring Boot 4.0.6, Spring Security 6, Spring Data JPA, Spring WebSocket (STOMP), OAuth2 Client
Veritabanı	PostgreSQL (H2 dev)
Auth	JWT (jjwt) + Google OAuth2
Frontend	React 18, Vite 5, Tailwind CSS, Alpine.js
3D / Animasyon	Spline, GSAP
Editör	Monaco Editor (@monaco-editor/react)
İşbirliği	Yjs CRDT + y-monaco + y-protocols
Terminal	xterm.js, WebContainer API (@webcontainer/api)
WebSocket	SockJS, @stomp/stompjs
HTTP	Axios
Build	Maven, Docker
Cloud	Render (Docker deploy)

Öğrenilen Kavramlar

- **Hybrid rendering**: Thymeleaf + React aynı uygulamada
- **Adjacency list**: Hiyerarşik veri modellemesi (parent_id)
- **STOMP over WebSocket**: Mesaj broker'lı pub/sub mimarisi
- **Yjs CRDT**: Çakışma olmadan gerçek zamanlı işbirlikçi metin düzenleme
- **WebContainer**: Tarayıcıda Node.js çalışma zamanı (WebAssembly)
- **JWT stateless auth**: localStorage + Authorization header
- **Multi-stage Docker**: Build ve runtime ayrımı

Slide 11: Demo Akışı (Önerilen)

1. **Landing Page** — Spline 3D animasyon, GSAP scroll trigger
2. **Register / Login** — Google OAuth2 ile giriş
3. **Dashboard** — Yeni oda oluştur, mevcut odaya katıl
4. **Room IDE** — Dosya oluştur, kod yaz, başka sekmede aynı odaya gir
5. **Real-time Sync** — İki tarayıcı penceresinde aynı dosyayı düzenle
6. **Run Code** — Python/JS kodunu çalıştır, terminal çıktısını göster
7. **Profile** — Profil fotoğrafı yükle, bilgileri güncelle

> **Teşekkürler!**
>
> Sorular?
>
> GitHub: github.com/Mowassir-Noor/CodeOrbit